

# Mac Mini M4 Pro — UE5 Setup & Build Guide

**Target Machine:** Mac Mini M4 Pro (Apple Silicon)  
**Project:** Final Evolution Lab  
**UE5 Version:** 5.4+  
**Total Estimated Time:** 4-6 hours (mostly waiting for downloads/builds)  
**Budget:** \$400/month AWS hosting

## Table of Contents

- 1. [System Requirements Check](#)
- 2. [Install Unreal Engine 5.4](#)
- 3. [Transfer the Project to Your Mac](#)
- 4. [Open the Project in UE5](#)
- 5. [Import AI-Generated Assets](#)
- 6. [Configure for Linux Packaging](#)
- 7. [Build/Package for Linux Server](#)
- 8. [Apple Silicon Optimizations](#)
- 9. [Performance Tips for M4 Pro](#)
- 10. [Troubleshooting](#)

## 1. System Requirements Check

⌚ **Time: 5 minutes**

Your Mac Mini M4 Pro should meet or exceed all requirements:

| Requirement | Minimum       | Mac Mini M4 Pro           | Status        |
|-------------|---------------|---------------------------|---------------|
| CPU         | 6 cores       | 12-core (M4 Pro)          | ✔ Exceeds     |
| RAM         | 16 GB         | 24/36/48 GB               | ✔ Exceeds     |
| GPU         | Dedicated GPU | Integrated 16/20-core GPU | ✔ Supported   |
| Disk Space  | 150 GB free   | 512GB-2TB SSD             | ✔ Check below |
| macOS       | Sonoma 14.0+  | Sequoia 15.x              | ✔             |
| Xcode       | 15.0+         | 16.x                      | ✔             |

### Verify Your System

Open **Terminal** (Cmd+Space → type “Terminal”) and run:

```
# Check macOS version
sw_vers

# Check available disk space (need ~150 GB free)
df -h /

# Check RAM
sysctl hw.memsize | awk '{print $2/1073741824 " GB"}'
```

```
# Check CPU cores
sysctl -n hw.ncpu

# Verify Apple Silicon
uname -m
# Should output: arm64
```

## Install Xcode Command Line Tools

```
xcode-select --install
```

⌚ This may take 10-15 minutes to download.

---

## 2. Install Unreal Engine 5.4

⌚ **Time: 45-90 minutes** (download + install)

### Step 2.1: Create Epic Games Account

1. Go to <https://www.unrealengine.com>
2. Click “**Download**” → “**Download Unreal Engine**”
3. Create an account or sign in
4. Accept the EULA

### Step 2.2: Download Epic Games Launcher

1. Go to <https://www.unrealengine.com/download>
2. Click “**Download for macOS**”
3. Open the downloaded .dmg file
4. Drag “**Epic Games Launcher**” to your **Applications** folder
5. Open **Epic Games Launcher** from Applications
6. Sign in with your Epic Games account

### Step 2.3: Install Unreal Engine 5.4

1. In Epic Games Launcher, click “**Unreal Engine**” tab on the left
2. Click “**Library**” at the top
3. Click the “**+**” button next to “ENGINE VERSIONS”
4. Select “**5.4.x**” from the dropdown (latest 5.4 patch)
5. Click “**Install**”
6. **Choose install location:** /Users/Shared/Epic Games/UE\_5.4
  - ⚠ **Needs ~45 GB** of disk space
7. Under **Options**, ensure these are checked:
  - ✓ Core Components
  - ✓ Starter Content
  - ✓ Templates and Feature Packs
  - ✓ **Target Platforms → Linux** (CRITICAL for server builds)
8. Click “**Apply**” then “**Install**”

⌚ Download is ~30 GB. On a 100 Mbps connection, expect ~40-60 minutes.

### Step 2.4: Link GitHub Account (Required for Linux Cross-Compile)

1. Go to <https://www.unrealengine.com/ue-on-github>
2. Click “**Connect**” next to GitHub
3. Authorize Epic Games on GitHub
4. Accept the invitation to the EpicGames GitHub organization

## Step 2.5: Verify Installation

```
# Check UE5 is installed
ls "/Users/Shared/Epic Games/UE_5.4/"

# Or if installed to default location
ls ~/Library/Epic\ Games/UE_5.4/

# Verify the Editor binary exists (Apple Silicon native)
ls "/Users/Shared/Epic Games/UE_5.4/Engine/Binaries/Mac/UnrealEditor.app"
```

---

## 3. Transfer the Project to Your Mac

🕒 **Time: 10-30 minutes** (depending on method)

See [PROJECT\\_TRANSFER\\_GUIDE.md](#) for detailed transfer instructions.

### Quick version:

```
# Create project directory
mkdir -p ~/Projects
cd ~/Projects

# Option A: Git clone (if repo is set up)
git clone https://github.com/YOUR_USERNAME/rork-final-evolution-lab.git

# Option B: Download ZIP from DeepAgent and extract
unzip ~/Downloads/rork-final-evolution-lab.zip -d ~/Projects/
```

---

## 4. Open the Project in UE5

🕒 **Time: 10-20 minutes** (first-time shader compilation)

### Step 4.1: Set Up Environment Variables

Create/edit the `.ue5_env` file for Mac:

```
cd ~/Projects/rork-final-evolution-lab

# Copy template
cp .ue5_env.template .ue5_env

# Edit for Mac paths
cat > .ue5_env << 'EOF'
# UE5 Mac Mini M4 Pro Configuration
export UE_ROOT="/Users/Shared/Epic Games/UE_5.4"
export UE_EDITOR="$UE_ROOT/Engine/Binaries/Mac/UnrealEditor.app/Contents/MacOS/UnrealEdit

export UE_CMD="$UE_ROOT/Engine/Binaries/Mac/UnrealEditor-Cmd"
export RUN_UAT="$UE_ROOT/Engine/Build/BatchFiles/RunUAT.sh"
export FEL_PROJECT="$(pwd)/UnrealStarter/BasketballGame/BasketballGame.uproject"

export PATH="$UE_ROOT/Engine/Binaries/Mac:$PATH"
EOF

# Load the environment
source .ue5_env
```

## Step 4.2: Open in Unreal Editor

### Method A: Double-click (Easiest)

1. Open **Finder**
2. Navigate to ~/Projects/rork-final-evolution-lab/UnrealStarter/BasketballGame/
3. Double-click **BasketballGame.uproject**
4. If prompted, select **UE 5.4** as the engine version

### Method B: Command Line

```
source ~/Projects/rork-final-evolution-lab/.ue5_env
open "$FEL_PROJECT"
```

### Method C: Via Epic Games Launcher

1. Open Epic Games Launcher
2. Go to **Unreal Engine** → **Library**
3. Under “My Projects”, click “**Browse**”
4. Navigate to ~/Projects/rork-final-evolution-lab/UnrealStarter/BasketballGame/
5. Select BasketballGame.uproject

## Step 4.3: First-Time Setup

When opening for the first time:

1. “**Would you like to convert this project?**” → Click “**Convert in Place**” if prompted
  2. **Shader compilation** will begin — this takes **10-20 minutes** on M4 Pro
    - You’ll see “Compiling Shaders (XXXX remaining)” in the bottom-right
    - The editor may feel slow until this completes
    - ☕ Grab a coffee!
  3. “**Missing Modules**” warning → Click “**Yes**” to rebuild
  4. Wait for the Content Browser to finish indexing
- 

## 5. Import AI-Generated Assets

🕒 **Time: 15-30 minutes**

### Step 5.1: Open the Python Console

In UE5 Editor: 1. Go to **Edit** → **Editor Preferences** → search “Python” 2. Enable “**Python Editor Script Plugin**” if not already enabled 3. Restart the editor if prompted 4. Go to **Window** → **Developer Tools** → **Output Log** 5. At the bottom of the Output Log, change the dropdown from “Cmd” to “**Python**”

### Step 5.2: Run Import Scripts

*# In the UE5 Python console, run each script:*

```
# 1. Import AI-generated assets (49 files)
exec(open('/Users/YOUR_USERNAME/Projects/rork-final-evolution-lab/UnrealStarter/BasketballGame/EditorPython/fel_import_ai_assets.py').read())
```

```
# 2. Import Elijah Bonds animations
```

```
exec(open('/Users/YOUR_USERNAME/Projects/rork-final-evolution-  
lab/UnrealStarter/BasketballGame/EditorPython/fel_import_elijahbonds_animations.f
```

```
# 3. Import catalogue animations
```

```
exec(open('/Users/YOUR_USERNAME/Projects/rork-final-evolution-  
lab/UnrealStarter/BasketballGame/EditorPython/fel_import_catalogue_animations.py')
```

Replace **YOUR\_USERNAME** with your macOS username (run `whoami` in Terminal to check).

### Step 5.3: Verify Assets

In Content Browser, check that these folders exist and contain assets:

- /Game/FEL/Generated/deepmotion/ — Animation assets -
- /Game/FEL/Generated/meshy/ — 3D model assets -
- /Game/FEL/Animations/ElijahBonds/ — Custom animations

## 6. Configure for Linux Packaging

🕒 **Time: 15-30 minutes** (one-time setup)

### Step 6.1: Install Linux Cross-Compile Toolchain

UE5 on Mac can cross-compile for Linux. You need the Linux cross-compile toolchain:

1. In UE5 Editor, go to **Edit** → **Project Settings**
2. Navigate to **Platforms** → **Linux**
3. If you see a “SDK not found” message, install it:

```
# The Linux cross-compile support should have been installed with  
UE5  
  
# Verify it exists:  
ls "/Users/Shared/Epic  
Games/UE_5.4/Engine/Extras/ThirdPartyNotUE/SDKs/"
```

If missing, re-run Epic Games Launcher → Library → UE 5.4 → Options → check “**Linux**” under Target Platforms.

### Step 6.2: Enable Required Plugins

1. In UE5 Editor, go to **Edit** → **Plugins**
2. Search and enable:
  - ✓ **Pixel Streaming** (under “Built-In” → “Media”)
  - ✓ **Pixel Streaming Player**
  - ✓ **Online Subsystem** (if using multiplayer)
3. Restart the editor when prompted

### Step 6.3: Configure Project Settings

1. **Edit** → **Project Settings**
2. **Project** → **Maps & Modes**:
  - Set **Default GameMode** to your FEL game mode
3. **Platforms** → **Linux**:
  - **Target Architecture**: x86\_64-unknown-linux-gnu
  - **Build Configuration**: Shipping (for production) or Development (for testing)
4. **Project** → **Packaging**:
  - **Build Configuration**: Shipping
  - Check “**Use Pak File**”

- Check “**Create compressed cooked packages**”
- Under “**Directories to always cook**”, add: /Game/FEL/

#### Step 6.4: Configure Pixel Streaming

1. **Edit** → **Project Settings** → search “Pixel Streaming”
  2. Set:
    - **Encoder:** Hardware (preferred) or Software
    - **Streamer ID:** DefaultStreamer
    - **Signalling Server URL:** ws://localhost:8888 (will be overridden at runtime)
- 

## 7. Build/Package for Linux Server

🕒 **Time: 1.5-3 hours** (packaging is CPU-intensive)

### Option A: Package from UE5 Editor (Recommended for First Build)

1. **File** → **Package Project** → **Linux (x86-64)**
  - If Linux doesn’t appear, go back to Step 6.1
2. Choose output directory: ~/Projects/rork-final-evolution-lab/Builds/Linux/
3. Wait for packaging to complete
  - 🕒 On M4 Pro: ~1.5-3 hours for first build
  - Monitor progress in the **Output Log** window

### Option B: Command Line Build (Advanced)

```
# Load environment
source ~/Projects/rork-final-evolution-lab/.ue5_env

# Package for Linux Server (Shipping)
"$RUN_UAT" BuildCookRun \
  -project="$FEL_PROJECT" \
  -noP4 \
  -platform=Linux \
  -clientconfig=Shipping \
  -serverconfig=Shipping \
  -cook \
  -build \
  -stage \
  -pak \
  -archive \
  -archivedirectory="$(pwd)/Builds/Linux" \
  -compressed \
  -utf8output \
  -installed
```

### Option C: Use the Cook Script

```
# Load environment
source ~/Projects/rork-final-evolution-lab/.ue5_env

# Run the cook script (modified for Mac)
chmod +x scripts/ue5_setup/cook_fel_linux_server.sh
bash scripts/ue5_setup/cook_fel_linux_server.sh --config Shipping
```

## Verify the Build

```
# Check build output exists
ls -la ~/Projects/rork-final-evolution-lab/Builds/Linux/

# Check the server binary exists
find ~/Projects/rork-final-evolution-lab/Builds/Linux/ -name
  "**Server*" -o -name "*.sh" | head -20

# Check approximate size (should be 2-8 GB)
du -sh ~/Projects/rork-final-evolution-lab/Builds/Linux/
```

---

## 8. Apple Silicon (M4 Pro) Optimizations

### Metal Rendering

UE5 uses **Metal** natively on Apple Silicon — no Rosetta needed for the Editor.

```
# Verify UE5 is running natively (not through Rosetta)
# While UE5 is open, run:
ps aux | grep -i unreal | grep -v grep
# Look for "arm64" in the process info
```

### Editor Performance Settings

1. **Edit → Editor Preferences** → search “Performance”
2. Set:
  - **Use Less CPU when in Background:** ✓ Enabled
  - **Editor Scalability:** High (M4 Pro handles this well)
3. **Edit → Project Settings → Engine → Rendering:**
  - **Default RHI:** Metal
  - **Mobile HDR:** Disabled (unless targeting mobile preview)

### Memory Management

```
# Close memory-heavy apps during packaging
# Monitor memory usage during build
vm_stat | head -5

# If you have 24 GB RAM, UE5 should work fine
# If you have 36/48 GB, you can increase parallel cooking
```

### Thermal Management

⚠ **Long builds will heat up the Mac Mini.** Ensure good ventilation.

- Place the Mac Mini in a well-ventilated area
- Don’t stack anything on top of it
- Consider a USB fan underneath for extended builds
- Monitor temperature during packaging:

```
# Install Intel Power Gadget alternative for Apple Silicon
brew install --cask hot
# Or use:
sudo powermetrics --samplers smc -i 5000 | grep -i temp
```

---

## 9. Performance Tips for M4 Pro

## Build Speed Optimization

| Tip                                    | Expected Improvement        |
|----------------------------------------|-----------------------------|
| Close other apps during packaging      | 10–20% faster               |
| Use SSD (built-in) not external HDD    | 2–3x faster I/O             |
| Connect to power (not battery)         | Sustained performance       |
| Set Energy Saver to “Never sleep”      | Prevents interrupted builds |
| Use wired internet for large downloads | More reliable               |

## System Preferences

```
# Prevent sleep during long builds
caffeinate -i -s &
# Kill with: kill %1

# Set Energy Saver (System Settings → Energy)
# Turn off "Put hard disks to sleep when possible"
# Set display sleep to 30+ minutes during builds
```

## Parallel Builds

The M4 Pro’s 12 cores can parallelize shader compilation and cooking:

```
# Set parallel jobs for builds (M4 Pro can handle 10)
export UE_PARALLEL_JOBS=10

# For Xcode builds
defaults write com.apple.dt.Xcode
    IDEBuildOperationMaxNumberOfConcurrentCompileTasks 10
```

## Storage Tips

```
# Check how much space UE5 uses
du -sh "/Users/Shared/Epic Games/UE_5.4/" # ~45 GB
du -sh ~/Projects/rork-final-evolution-lab/ # ~5–10 GB

# After successful build, clean intermediate files
rm -rf ~/Projects/rork-final-evolution-lab/UnrealStarter/BasketballGame/Intermediate/
rm -rf ~/Projects/rork-final-evolution-lab/UnrealStarter/BasketballGame/Saved/
# Reclaim ~10–30 GB
```

# 10. Troubleshooting

## “Project was made with a different version of UE”

**Solution:** Click “Convert in Place” or “Open a Copy”. The project is compatible with UE5 5.3–5.5.

## Shader Compilation Takes Forever

**Expected Behavior:** First open compiles 5,000–15,000 shaders. On M4 Pro this takes 10–20 minutes.

### If stuck:

```
# Clear shader cache and try again
```



```
rm -rf ~/Projects/rork-final-evolution-  
lab/UnrealStarter/BasketballGame/Saved/ShaderCache/  
rm -rf ~/Library/Caches/com.epicgames.UnrealEditor/
```

## Linux Cross-Compile Fails

**“Can’t find Linux toolchain”:** 1. Open Epic Games Launcher → Library → UE 5.4 2. Click the dropdown arrow → Options 3. Check **“Linux”** under Target Platforms 4. Click Apply/Update

**“clang not found”:**

```
xcode-select --install  
sudo xcode-select --switch /Applications/Xcode.app
```

## Build Fails with “Out of Memory”

```
# Close Chrome and other memory-heavy apps  
# Increase swap space (temporary)  
sudo sysctl vm.swapusage
```

```
# Reduce parallel jobs  
export UE_PARALLEL_JOBS=6
```

## UE5 Editor Crashes on Launch

```
# Delete cached data  
rm -rf ~/Library/Preferences/com.epicgames.*  
rm -rf ~/Library/Application\ Support/Epic/  
rm -rf ~/Projects/rork-final-evolution-  
lab/UnrealStarter/BasketballGame/Saved/  
  
# Try launching from command line to see errors  
"/Users/Shared/Epic  
Games/UE_5.4/Engine/Binaries/Mac/UnrealEditor.app/Contents/MacOS/UnrealEditor"  
\  
  
~/Projects/rork-final-evolution-  
lab/UnrealStarter/BasketballGame/BasketballGame.uproject \  
-log
```

## Metal API Errors

```
# Update macOS to latest version  
softwareupdate --list  
softwareupdate -i -a  
  
# Reset Metal shader cache  
rm -rf ~/Library/Caches/com.apple.metal/
```

## Build Takes Too Long

Expected build times on Mac Mini M4 Pro:

| Task                                | Estimated Time |
|-------------------------------------|----------------|
| UE5 Download & Install              | 45–90 min      |
| First project open (shader compile) | 10–20 min      |
| Full Linux package (first time)     | 1.5–3 hours    |
| Incremental rebuild                 | 15–45 min      |
| iOS package                         | 30–60 min      |

☞ **Pro Tip:** Start a build before bed. Use caffeineate to prevent sleep, and the build will be ready in the morning.

```
# Start build and prevent sleep
caffeinate -i -s bash -c 'source .ue5_env && "$RUN_UAT" BuildCookRun \
    -project="$FEL_PROJECT" -platform=Linux -clientconfig=Shipping \
    -cook -build -stage -pak -archive -
    archivedirectory="$(pwd)/Builds/Linux" '
```

---

## Next Steps

Once your build is complete:

1. 📁 **Upload to AWS** → See [MAC\\_AWS\\_DEPLOYMENT.md](#)
  2. 🚀 **Deploy to EC2** → See [DEPLOYMENT\\_CHECKLIST.md](#)
  3. 🎧 **Test the stream** → See [TESTING\\_GUIDE.md](#)
  4. 📱 **Build for iOS** → See [scripts/ue5\\_setup/cook\\_fel\\_ios.sh](#)
- 

*Last updated: April 2, 2026*

*Optimized for Mac Mini M4 Pro with Apple Silicon*